MECHTRON / SFWRENG 3K04: Software Development

Project Best Practices – Fall 2026

McMaster University

Introduction

With this document, we aim to provide some practical recommendations to help teams successfully develop the course project. These guidelines are intended to complement the project specifications by highlighting some good software engineering practices, technical considerations, and collaboration strategies that can help improve the software quality and facilitate the final integration between both projects (i.e., systems). While there are some terms you are not familiar with yet, or concepts you do not know how to apply yet, make sure you revisit this document throughout the semester.

This document covers the following topics:

1. General software engineering best practices
2. Team communication and collaboration
3. Project parts-specific development recommendations
   A. **Part A - Automated Dispensing Cabinet (ADC):** The embedded software that controls a secure, software-driven cabinet at the point of care. It handles user authentication, dispensing, inventory, event logging, fault detection, operational modes, and communication with an external monitor.
   B. **Part B - Hospital Monitoring System:** The supervisory, human-facing application that talks to one or more ADCs. It handles ADC communication, inventory supervision, alarm management, historical data, reporting, dashboards, and cabinet-health monitoring.
4. Implementation and testing recommendations
5. Interface design and integration considerations

1 General Software Engineering Best Practices

Successful software projects begin with a good understanding of the problem rather than implementation. It is suggested to spend more time on this activity instead of writing the first lines of code. Then, you and your team are encouraged to approach any of the projects assigned to your team as a short well-controlled capstone project that requires requirements elicitation, analysis, design, implementation, testing, integration, and documentation. The goal is to develop a well-structured solution developed incrementally than a large implementation developed without sufficient planning.

Recommendations:

- Read and understand the project specification multiple times before starting any implementation and discuss it with your peers first.

- Clearly identify the project scope, stakeholders and their responsibilities, system boundaries, and document any assumptions made during the development process.
- Identify functional and non-functional requirements; and design the software architecture before starting the implementation.
- Decompose the system into cohesive modules with clear responsibilities.
- Perform hazard analysis on the system to identify safety/security issues so that you can come up with a set of safety requirements.
- When it comes to implementation, use meaningful names for classes, methods, variables, and files.
- Keep implementations simple, modular, and maintainable. This includes avoiding hard-coded values whenever possible.
- Always document important design decisions and justify implementation choices.
- Use version control like GitHub throughout the project and commit changes regularly.

2 Team Communication and Collaboration

Software engineering, like any other engineering discipline, is a collaborative activity. Good and regular communication is essential for the success of the project execution. As a team, you should regularly discuss assumptions, constraints, implementation decisions, and interface changes to minimize misunderstandings and rework.

Recommendations:

- Hold regular team meetings to review progress and discuss issues to be addressed as they arise.
- Document and communicate assumptions and important engineering decisions instead of relying on implicit understanding. This includes keeping project documentation up to date.
- Divide work according to software components and responsibilities.
- Integrate completed work frequently instead of waiting until the end of the project.
- Discuss interface and integration issues early with sub-teams (i.e., communication between sub-teams working on Projects A and B).

We encourage you to complete the team contract, which is 1% of your grades, to make sure you set up expectations around communication and work ethic before starting the project.

3 Project Part-Specific Development Recommendations

Part A Automated Dispensing Cabinet (ADC)

The ADC part of the project focuses on the development of a piece of software that controls a safety-critical device. The primary challenge is not implementing hardware functionality but

ensuring that software makes safe decisions before performing operations that could affect patients or the physical state of the cabinet. Thus, your team should focus on behavioural correctness, state management, and fault handling rather them realistic hardware simulation.

Recommendations:

- Model the operational behaviour of the ADC before implementation.
- Define operational states and valid state transitions.
- Treat sensors and actuators as software interfaces rather than physical devices.
- Perform safety/security checks before executing medical dispensing operations.
- Handle abnormal situations and fault conditions.
- Record important operational events and fault conditions.

Part B Hospital Monitoring System

Unlike ADC, think of the Monitoring System as a supervisory application coordinating multiple ADC devices. The emphasis should be on maintaining a consistent view of the system, including management of workflows, validation of the information, and supporting operational decision making. Your team should focus on designing a piece of software that remains reliable even when information is incomplete or communication problems occur.

Recommendations:

- Design the software around operational workflows rather than individual screens of menus.
- Design the system to support multiple ADC units without assuming a fixed number of devices.
- Separate responsibilities like user management, communication, reporting, and data management.
- Validate requests before coordinating operations with ADC systems.
- Handle situations like unavailability of ADCs and communication failures.
- Maintain consistent and traceable information across the system.
- Record important operational events and user actions for later review.
- Avoid unnecessary implementation complexity, like databases and advanced dashboards.

4 Implementation and Testing Recommendations

Good implementation progress should proceed incrementally, with testing performed throughout development. You team is encouraged to verify individual modules, test complete operational scenarios, and include normal and abnormal situations. Since both Parts A and B represent safety-critical systems, testing activities should ideally demonstrate not only correct functionality but also appropriate behaviour when faults occur.

Recommendations:

- Implement the software in small, testable increments.
- Verify each module before integrating it with the rest of the system.
- Create test cases for both normal and abnormal scenarios.
- Test both normal and failure situations.
- Verify system behaviour after every major implementation milestone.
- Reserve sufficient time for debugging, integration, and final validation of the solution.
- Focus on software quality, robustness, and maintainability rather than implementation size.

5 Interface Design and Integration Considerations

Parts A and B of the project will be developed independently and integrated for the last deliverable. For this reason, interface design should be considered early rather than left until implementation is complete. As engineers, you should learn that well-defined interfaces reduce integration problems and help teams to think carefully about system responsibilities and communication requirements.

Recommendations:

- Define communication interfaces before implementation begins.
- Agree on exchanged information, message formats, and expected responses with the other sub-team.
- Clearly define which system owns each responsibility.
- Treat interfaces as contracts between independently developed systems.
- Test communication incrementally instead of waiting until the end of the semester.
- Validate received information before processing it.
- Notify the other sub-team whenever interface changes become necessary as early as possible.